

操作系统

(Operating Systems, OS)

主 讲： 吕慧英

单 位： 计算机科学与技术学院

联系方式： huiying@jlu.edu.cn

QQ 群： 1065942058



前言

先修课程：计算机组成原理；微机原理与汇编程序设计；计算机系统结构；高级语言程序设计；数据结构

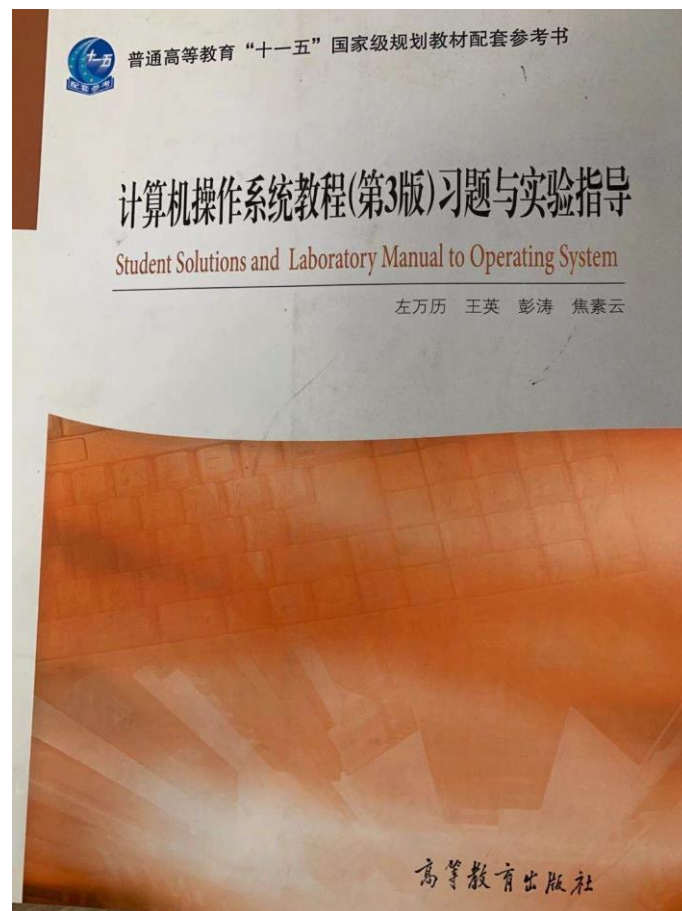
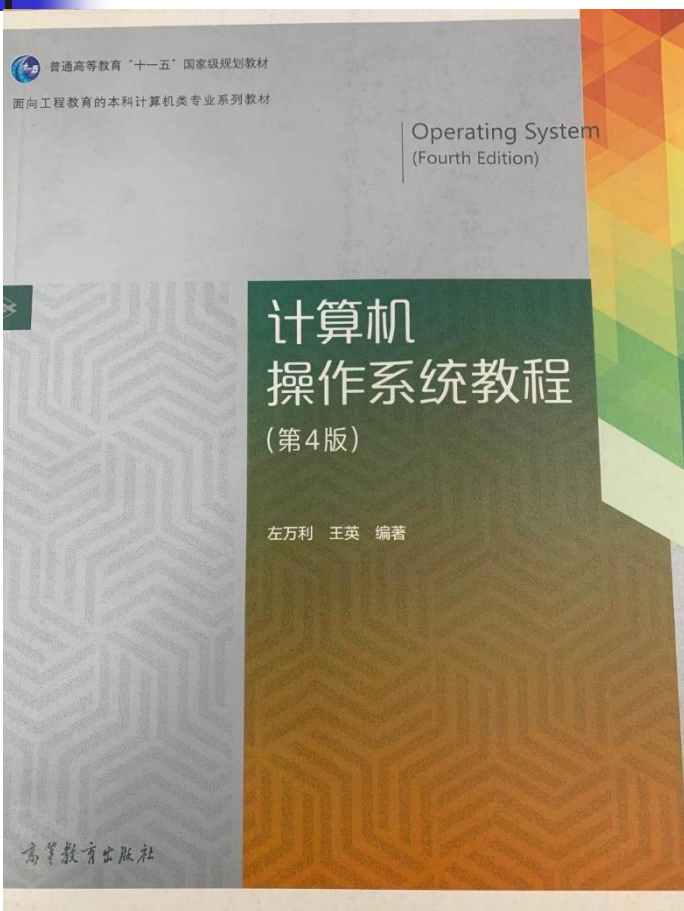
课程性质：必修的专业基础课程。

课程内容：操作系统与硬件和其它软件的关系；
操作系统对计算机系统中的硬、软件资源进行管理；
操作系统的基本结构、工作原理及实现方法。

开课目的：掌握操作系统的原理，
为分析和设计一个操作系统打下理论基础；
为开发其它软件系统打下理论基础。

课程支撑指标：2-2、3-1、3-2、4-1

教材：《计算机操作系统教程》（第4版）左万历、王英





课程形式

- 主课+实验课（64学时）
 - 主课48学时（第1周-12周）
 - 实验24学时（第10周-15周）
- 考试成绩
 - 期末试卷（60%）
 - 平时（10%）
 - 实验（30%）



参考资料

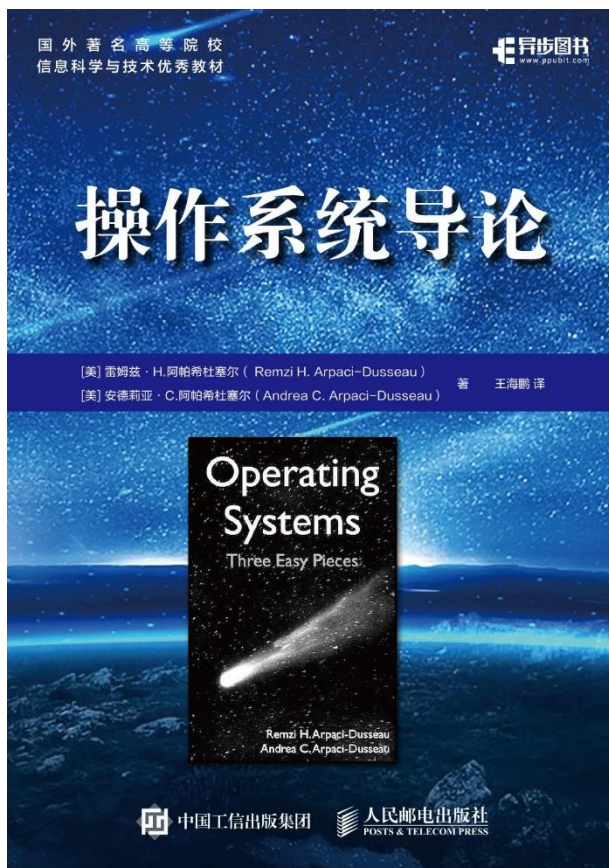
- A. Silberschatz, P. Galvin, **Operating System Concepts**, 10th edition, Wiley, 高等教育出版社, **2023**.
 - 系统，完善，国外大学多选用
- A. S. Tanenbaum. **Modern Operating Systems**, 5rd edition, Prentice Hall, 机械工业出版社, 2025.
 - 国内用的比较多
- William Stalling, **Operating Systems, Internals and Design Principles**, 9th Edition, Prentice Hall, 2022.
 - 另一本比较权威的教材



参考文献

- H.M. Deitel, P.J.Deitel, D.R. Choffnes. ***Operating Systems***, 3rd edition, 施平安等译, 清华大学出版社, 2007.
 - 很完整, 篇幅很长 (1331页)
- 孙钟秀等, ***操作系统教程***, 第4版, 高等教育出版社, 2008.4
 - 国内代表性教材
- ***莱昂氏UNIX源代码分析***, 6th edition, 机械工业出版社, 2001.
 - UNIX源代码10000行
 - C语言9000行, 汇编语言1000行
 - PDP11-45 , 要求了解硬件体系结构, 指令系统

参考文献

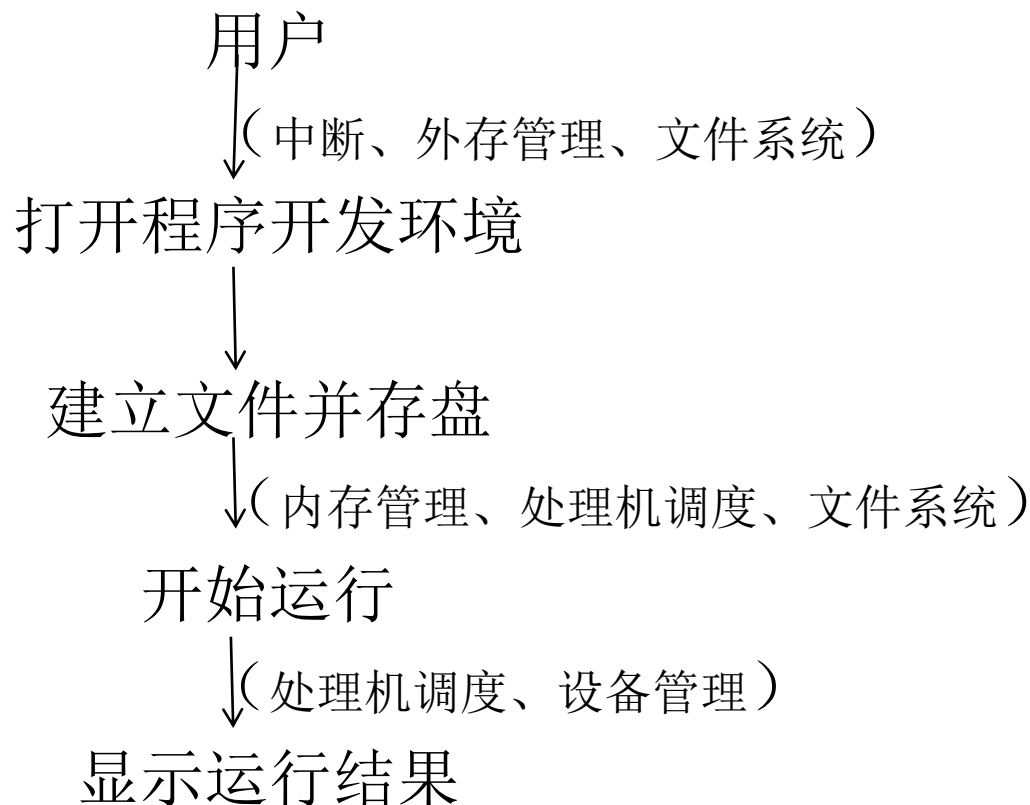


- 由浅入深介绍现代操作系统经典理论与方法
- 结合前沿研究与工业界实践，面向真实场景与真实问题
- 全新打造ChCore实验内核，建立对操作系统的第一手实践经验

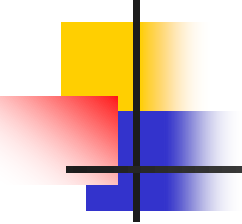
机械工业出版社
China Machine Press

认识操作系统

■ 用户使用计算机的过程



```
1 #include <stdio.h>
2
3 int main() {
4     printf("Hello, World!\n");
5     return 0;
6 }
```

教材内容结构

- 第一章 操作系统概述
- 第二章 进程、线程与作业
- 第三章 中断和处理器调度
- 第四章 互斥、同步与通信
- 第五章 死锁与饥饿
- 第六章 主存储管理
- 第七章 虚拟存储系统
- 第八章 文件系统
- 第九章 设备与输入输出管理
- 第十章 操作系统管理*
- 第十一章 操作系统设计*
- 第十二章 操作系统新技术*
- 第十三章 UNIX实例分析



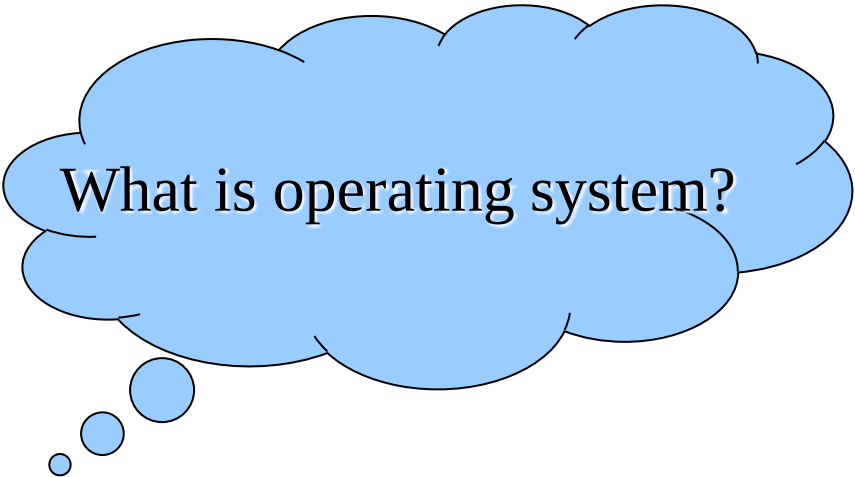
第一章 操作系统概述

- 操作系统的概念
- 操作系统的历史
- 操作系统的特性
- 操作系统的分类
- 操作系统的硬件环境
- 操作系统的界面形式
- 操作系统的运行机理
- 研究操作系统的几种观点



1.1 操作系统概念

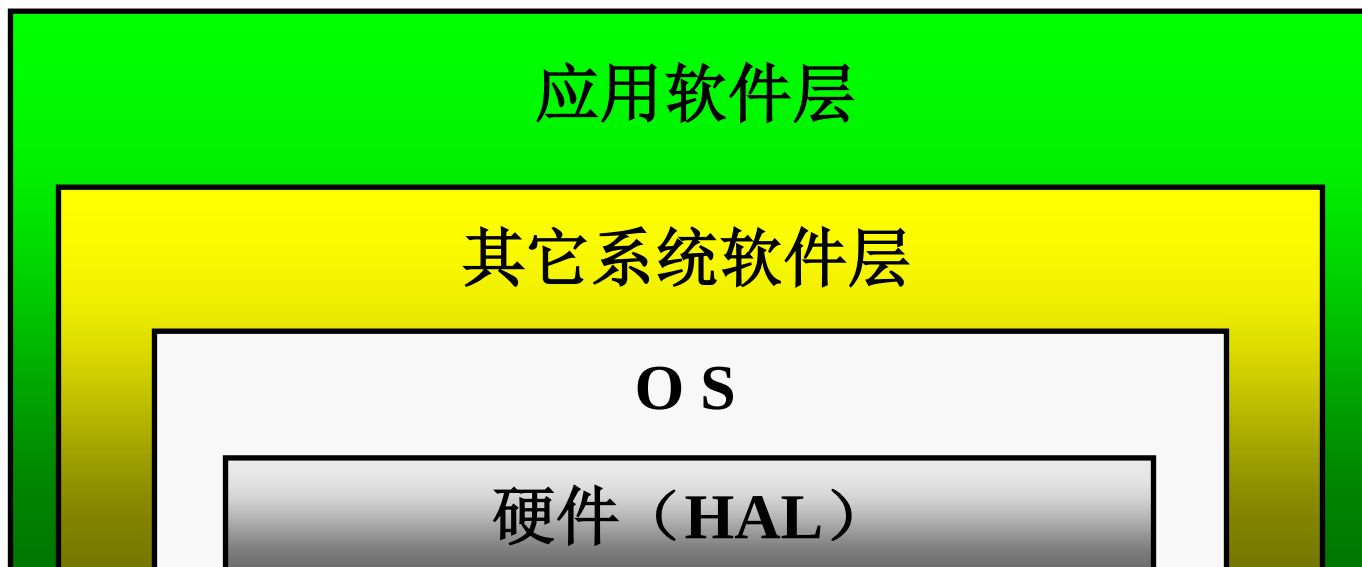
- 操作系统地位
- 操作系统作用
- 操作系统定义



What is operating system?

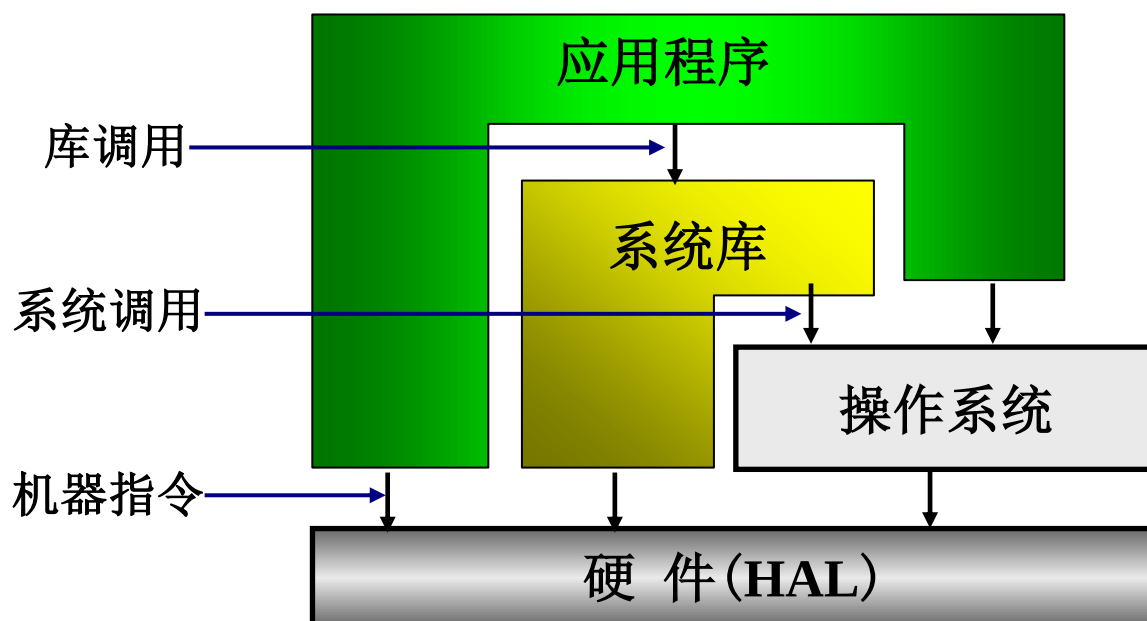
1.1.1 操作系统地位

- 硬件抽象层（**HAL**）之上
- 所有其它软件层之下



运行视图

- 系统库(**lib**)可调用操作系统，执行硬件指令
- 应用程序可以调用**lib**和操作系统，执行硬件指令





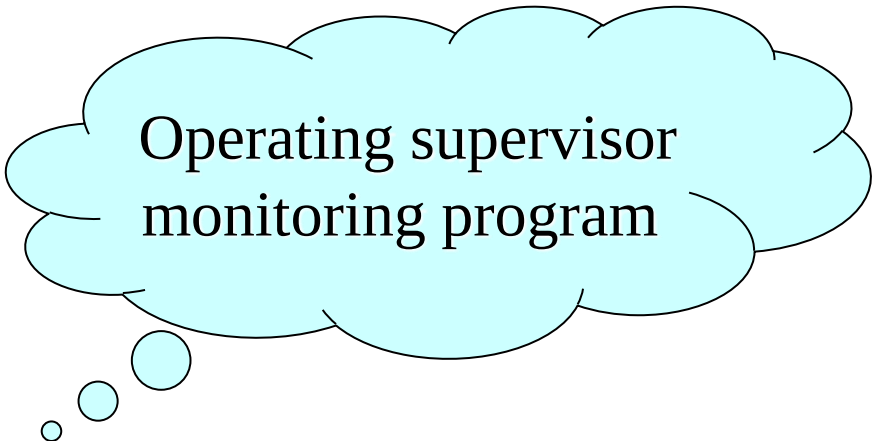
1.1.2 操作系统的作用

- 管理系统中软件硬件资源
 - **CPU:** 一个**CPU**, 多个可运行的程序
 - 内存: 进程空间相对独立, 支持共享
 - 设备: 分配, 驱动
 - 文件: 实现文件系统, 支持文件操作
- 为用户(应用程序)提供良好的服务(界面)
 - **API**
 - **GUI**, 行式命令(**ls, cd, cat, vi, rm, mount, ...**)
 - **JCL (Job Control Language)**

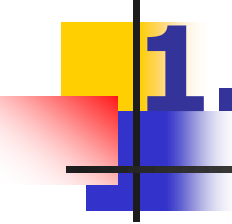


1.1.3 操作系统定义

- 操作系统是位于硬件层(**HAL**)之上，所有其它软件层之下的一个系统软件，通过它管理系统中各种软硬件资源，使它们被充分利用，方便用户使用计算机系统。



Operating supervisor
monitoring program



1.2 操作系统的历史

- 操作系统的产生

- 手工操作阶段
- 成批处理阶段
- 执行系统阶段

- 操作系统的完善

- 多道批处理系统
- 分时系统
- 实时处理系统
- 通用操作系统

- 操作系统的发展

- 网络操作系统
- 分布式操作系统
- 多处理机操作系统
- 单用户操作系统
- 面向对象操作系统
- 嵌入式操作系统
- 智能卡操作系统
- 多核技术下新一代操作系统

Evolution

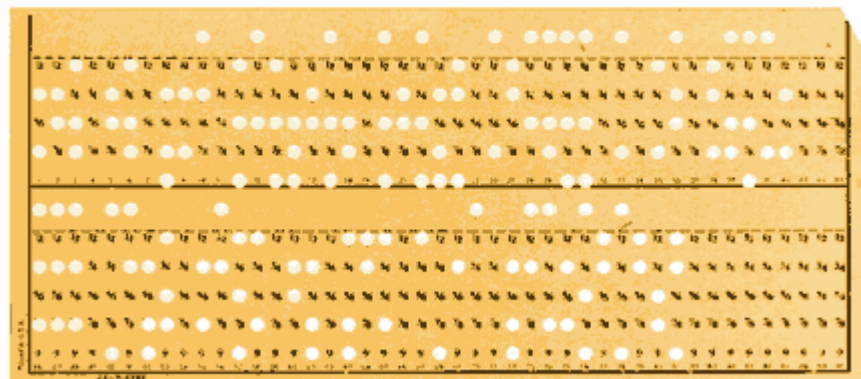
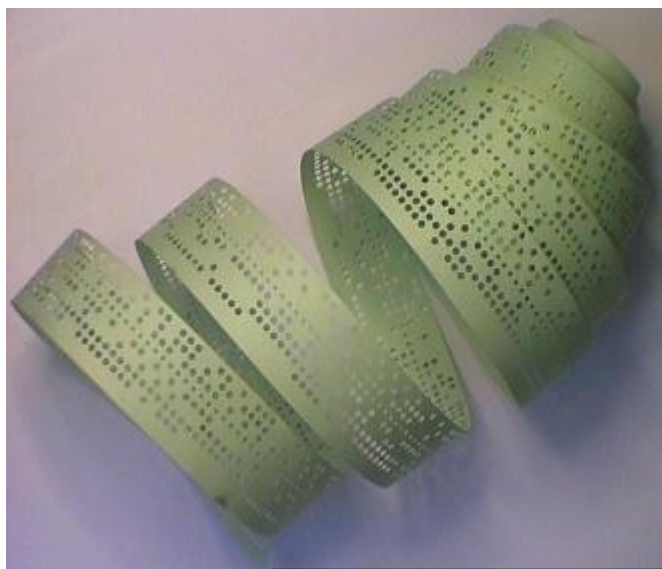
1.2.1 操作系统的产生

1、手工操作阶段（20世纪40年代，无操作系统）



1.2.1 操作系统的产生

- 电子管时代（1946-1955）
- 特点：
 - 硬件：电子管、接线面板（按钮/开关）；
 - 程序：二进制程序，打孔：纸带和卡片
 - 应用：科学计算





1.2.1 操作系统的产生

作业处理步骤:

(程序、数据)→穿孔机→纸带;
穿孔纸带→光电机→机器内存;
控制台开关启动第一条指令
(出错时显示地址,修改指令);
(如有输入需要安装数据纸带);
运行结果在电传打字机上输出。

缺点:

效率低: CPU有效运行时间极低

资源独占

程序准备/启动/结束: 手工处理, 繁琐耗时



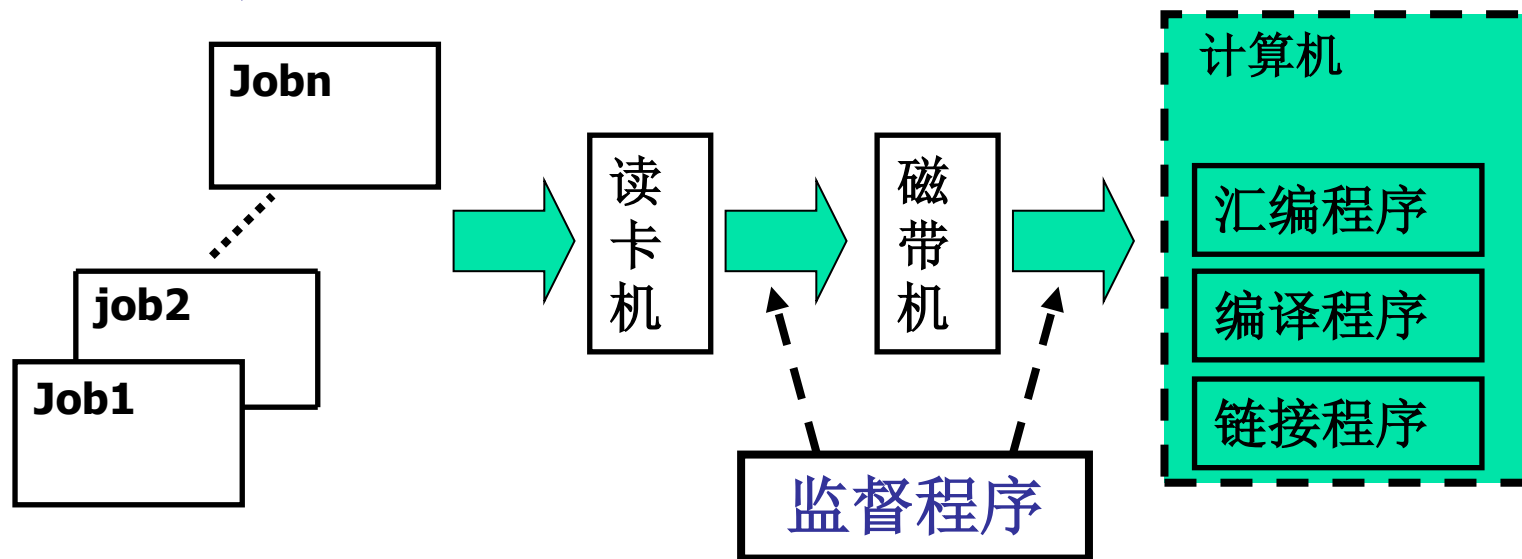
例子

- 一个作业在**1000次/s**的机器上运行需要 **1 hour**，手动操作时间**4min**，手动时间与程序运行时间之比为**1:15**；（第一代电子管）
- 如果计算机速度提高到**600,000次/s**，同样程序运行只需**6s**，而手动操作时间不变，手动操作与程序运行时间之比为**40:1**（第二代晶体管）

1.2.1 操作系统的产生(Cont.)

2、批处理阶段（20世纪50年代，操作系统雏形）

(1) 联机批处理：



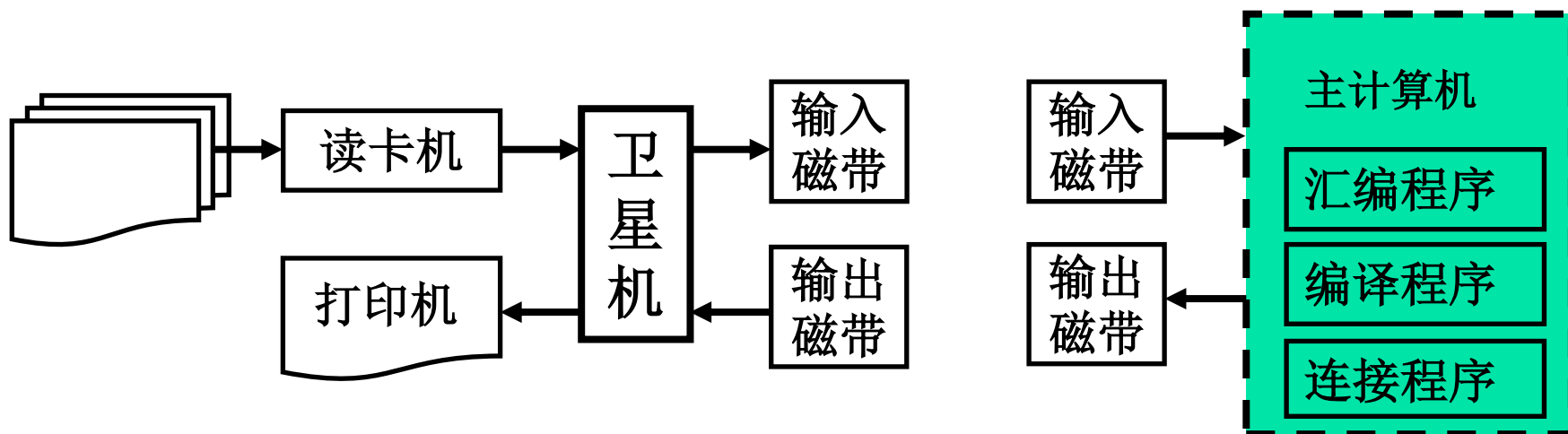
优点：摆脱了人工干预（作业过渡没有人的干预,一个作业处理过程没有人的干预）

缺点：**I/O**操作慢，主机等待时间长

1.2.1 操作系统的产生(Cont.)

2、批处理阶段

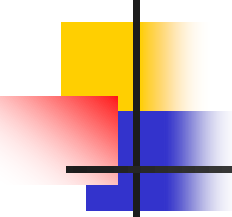
(2) 脱机批处理:



优点: 减少了主机等待**I/O**操作时间

缺点: **(1)**人工搬动磁带; **(2)**额外的卫星机

标准**I/O**程序, 编译程序, 连接装配程序出现



1.2.1 操作系统的产生(Cont.)

3、执行系统阶段（20世纪60年代初期）

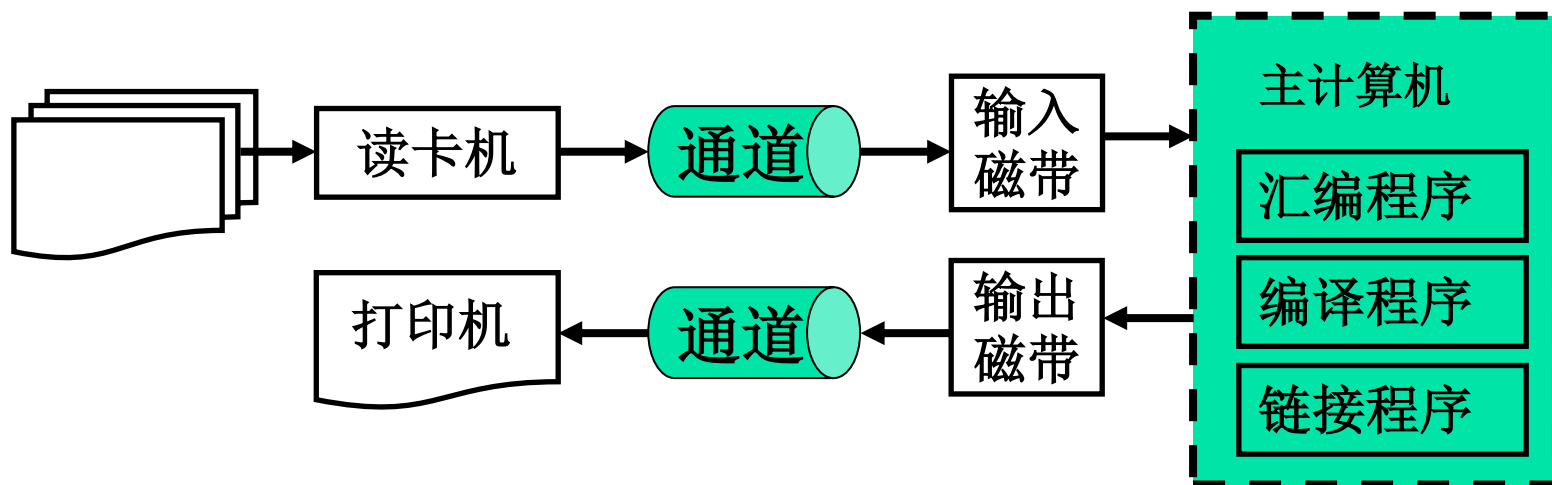
通道：专门用于控制**I/O**设备的处理机，即**I/O**处理机。

- 有自己的指令系统和运控部件；
- 与主机共享内存；
- 接受主**CPU**委托执行通道程序，完成**I/O**；
- 通道**I/O**操作与主**CPU**并行；
- 通道完成**I/O**时，向主机发中断请求。

操作系统的初级阶段，通道和中断技术的出现。

1.2.1 操作系统的产生(Cont.)

3、执行系统阶段



非联机, 非脱机,

假脱机(SPOOLing, Simultaneous Peripheral Operation On-Line)



1.2.2 操作系统的完善

1. 多道批处理系统（60年代初期）

- 执行系统：单道作业，资源利用不充分；
- 多道批处理：主机中同时放多个作业，最大限度提高资源利用率；
 - 单道到多道：不是量的变化，是质的飞跃
 - 带来问题：互斥、同步、通讯、死锁、饥饿、饿死
 - 多道批处理出现，标志操作系统走向成熟
 - 适合于大型科学计算任务



1.2.2 操作系统的完善

2. 分时系统（60年代初、中期）

- 程序员提出:对于批处理操作系统, 程序员无法了解作业的执行情况并对其进行动态控制
- 由一台主机和若干台与其相连的终端构成, 系统采用对话方式为各台终端上的用户服务
- 便于程序的动态修改和调试, 缩短了程序的处理周期
- 适合于交互式任务



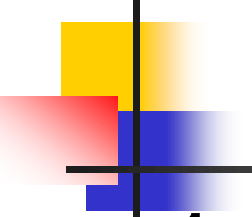
1.2.2 操作系统的完善

3. 实时系统（60年代中期）（第三代集成电路）

- 背景: 计算机应用领域扩大: (工业控制、医疗控制、航班订票等)。
- 要求: 满足时间约束条件

4. 通用操作系统（60年代后期）

- 上述三类系统的结合, 标志操作系统完善。
- 通用操作系统可以同时处理实时任务、接受终端请求、运行成批作业。
- 通用操作系统更加庞大、更加复杂, 造价也更高。



1.2.3 操作系统的发展

1. 计算机体系结构由集中向分散的发展，出现了计算机网络，由此产生网络操作系统和分布式操作系统；
2. 随着家用和商用微型计算机的普及，出现了单用户多任务的操作系统；
3. 大型计算任务要求计算机系统具有极强的计算和处理能力，产生了支持多处理器的并行操作系统；
4. 随着各种处理器芯片和存储介质在控制领域的广泛应用，出现了微内核(micro kernel)操作系统体系结构，产生了嵌入式和智能卡操作系统；
5. 伴随后摩尔时代的到来，提高单处理器速度已近极限，多核技术应运而生。新一代操作系统遇到的问题：多核的并发控制；多核下的进程调度。



1.2.3操作系统的发展

- **6.**随着集群系统和云计算的出现，产生了集群操作系统和云计算操作系统，它们都属于并行操作系统的范畴；



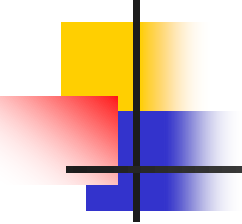
1.3 操作系统特性

- 并发性

- 多个程序在宏观上同时向前推进
- 并发(**concurrent**) vs. 并行 (**parallel**)
 - 用户程序与用户程序并发
 - 用户程序与**OS**并发
 - **OS**与**OS**并发

- 共享性

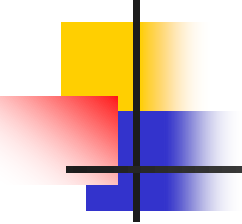
- 多个程序共用系统中的各种软硬件资源
- 在操作系统的协调和控制下



1.3 操作系统特性(Cont.)

- 程序异步性

- 宏观上同时运行多个程序（包括用户和操作系统程序）；
- 微观上多个程序（连同操作系统程序）交替运行；
- 交替的切换点是中断：
 - 用户程序向操作系统切换；
 - 操作系统程序向操作系统程序切换（中断嵌套）。
- 中断随机发生，致使程序切换不确定、不可预知。



1.3 操作系统特性(Cont.)

- 虚拟性

- 所谓虚拟就是利用某种技术把一个物理实体变为若干个逻辑实体；

例如：多道程序系统把单**CPU**系统中一个**CPU**改造成多个虚**CPU**；

虚拟存储管理技术扩充内存空间；

虚拟设备管理把独占型设备改造成共享的设备。



1.4 操作系统类型

- 多道批处理操作系统(batch processing system)
- 分时操作系统(time-sharing system)
- 实时操作系统(real time system)
- 通用操作系统(multi-purpose system)
- 单用户操作系统(single user system)
- 网络操作系统(network operating system)
- 分布式操作系统(distributed operating system)
- 多处理机操作系统(multi-processor system)
- 集群操作系统 (cluster operating system)
- 云计算操作系统(cloud computing operating system)



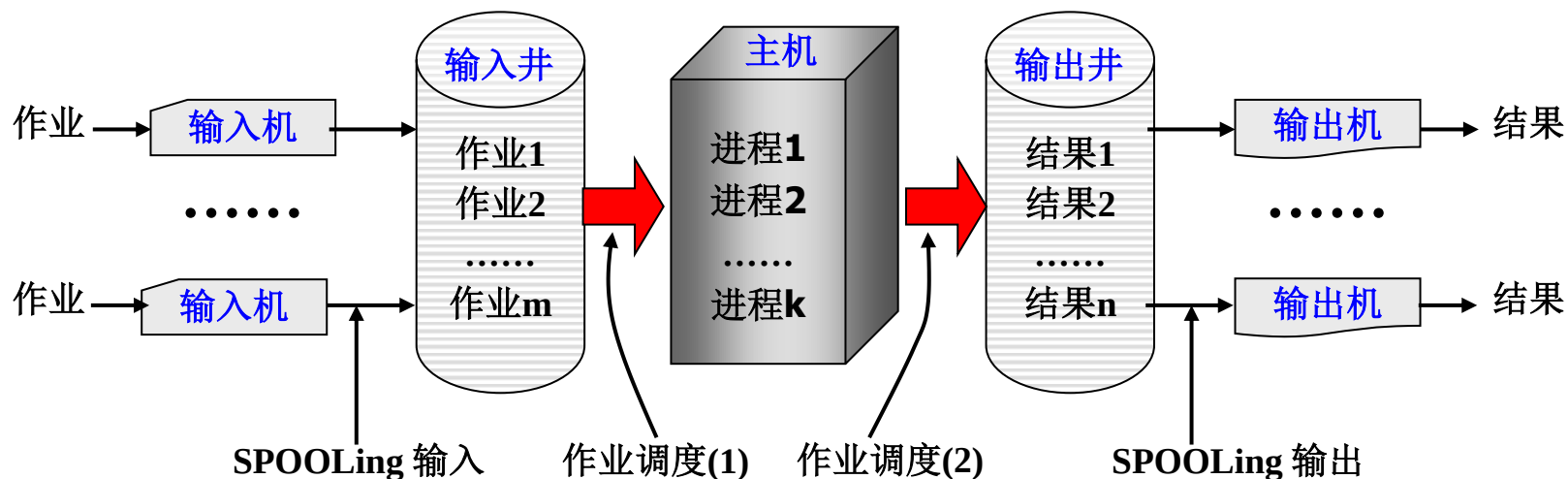
1.4 操作系统类型

- 嵌入式操作系统(**embedded operating system**)
- 多媒体操作系统(**multi-media operating system**)
- 智能卡操作系统(**smart-card operating system**)

1.4.1 多道批处理系统(off-line)

作业 (Job): 程序 + 数据 + 说明书 (JCL编写)

结果: 程序运行结果 + 记帐信息



多道批处理系统工作原理



1.4.1 多道批处理系统(cont.)

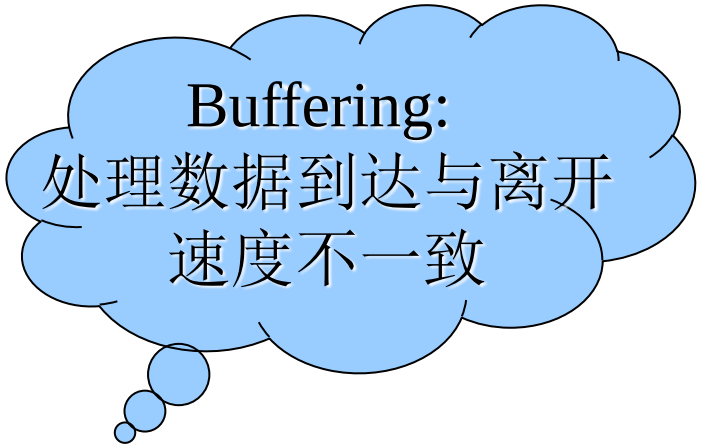
SPOOLing (*Simultaneous Peripheral Operation On-Line*)

- 称为假脱机工作方式或排队转储技术;
- 通过通道, 把**I/O**工作脱机处理, 但是在主机控制下运行, 故称为假脱机技术;
- 由专门负责**I/O**的**常驻内存进程**以及输入井、输出井组成;
- 将独占设备改为共享设备, 实现了虚拟设备功能。



1.4.1 多道批处理系统(cont.)

- 输入井作用
 - 缓冲(速度匹配作用)
 - 实现作业调度(**job scheduling**)
- 输出井作用
 - 缓冲(速度匹配作用)



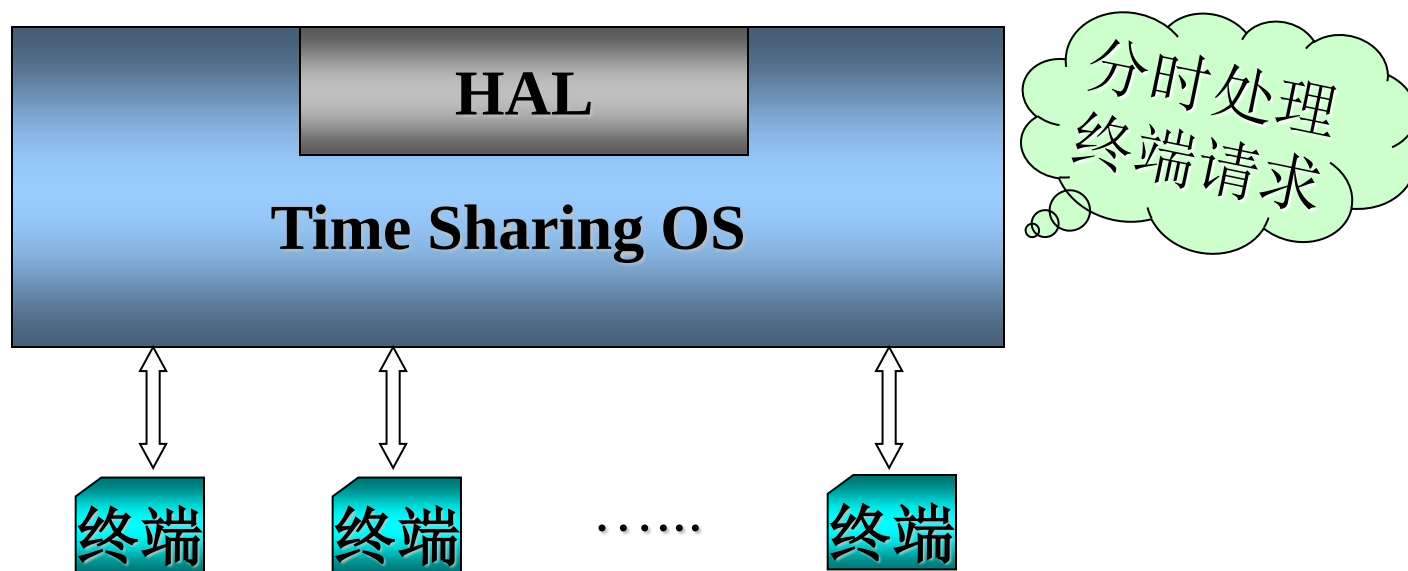
Buffering:
处理数据到达与离开
速度不一致



1.4.1 多道批处理系统(Cont.)

- 主机中作业合理搭配
 - 目标1: 提高资源利用率(eg. 计算型+IO型)
 - 目标2: 提高吞吐量(throughput)
- 特点
 - 多道: 系统中同时容纳多个作业
 - 成批: 作业分批进入系统

1.4.2 分时操作系统 (On-line)



界面1: 交互式命令语言 (eg. shell, command)

界面2: 图形用户界面 (GUI)



1.4.2 分时操作系统(Cont.)

- 特点:

- 多路性: 一个主机与多个终端相连;
- 交互性: 以对话的方式为用户服务;
- 独占性: 每个终端用户仿佛拥有一台虚拟机。

- 典型系统:

- **CTSS(MIT)**
- **Multics (MIT)**
- **UNIX**



1.4.3 实时操作系统

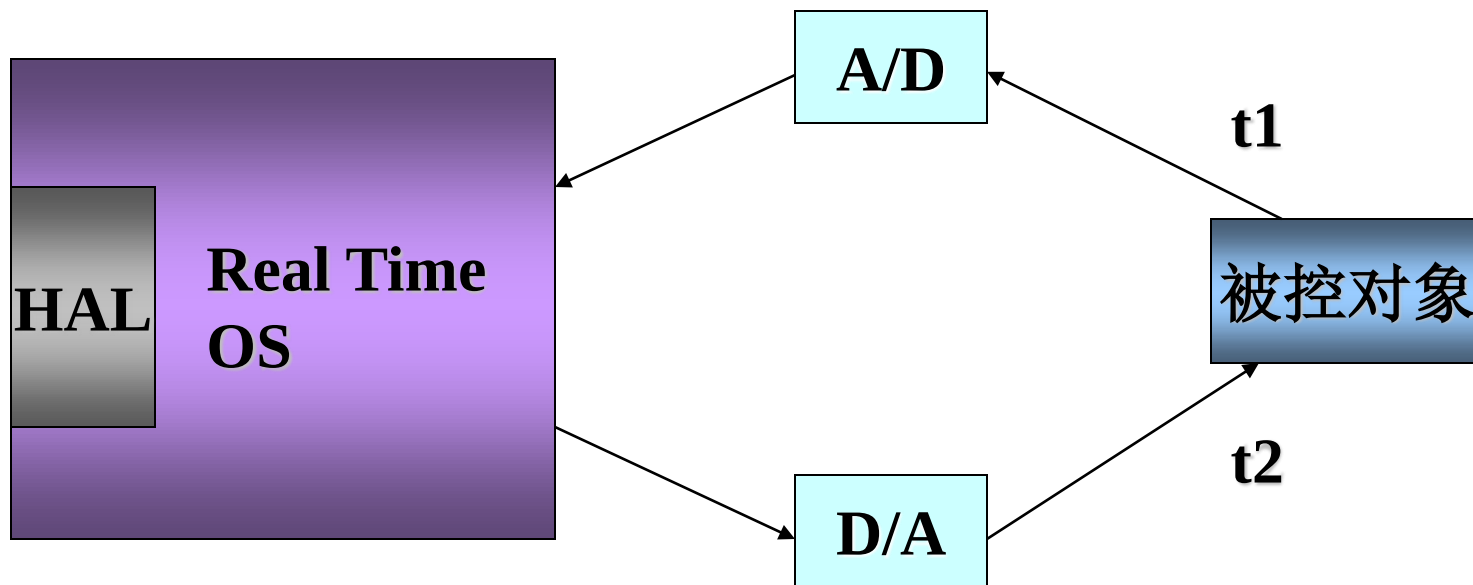
实时控制

- 工业控制，军事控制，医疗控制，.....

■ 实时信息处理

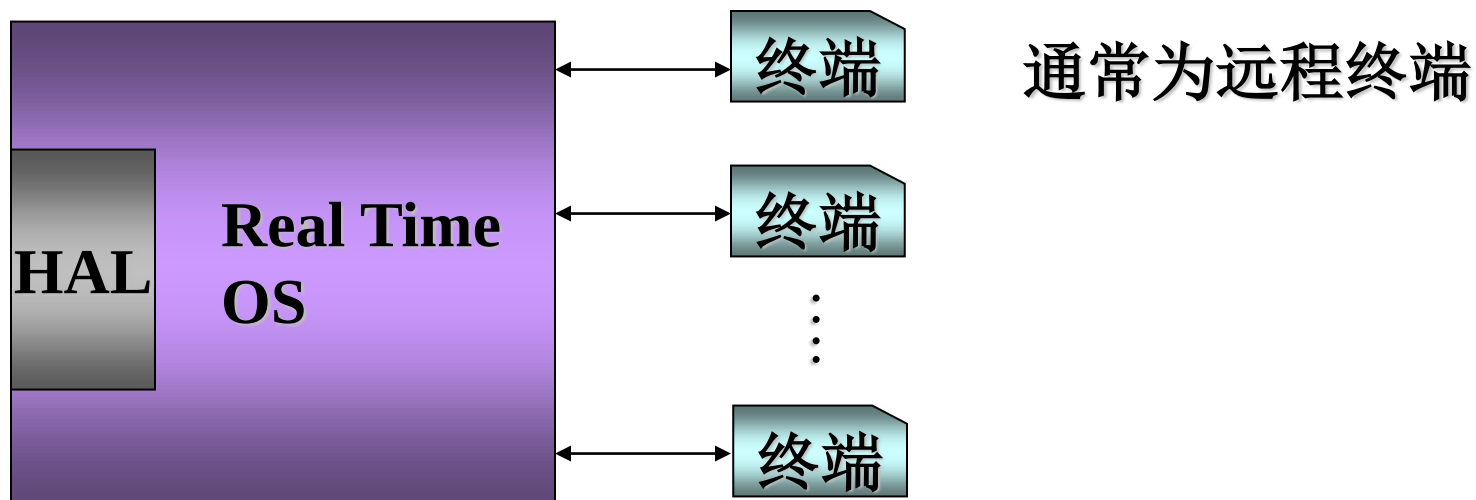
- 航班定票，联机情报检索，.....

实时控制



$t_2 - t_1$: response time

实时信息处理

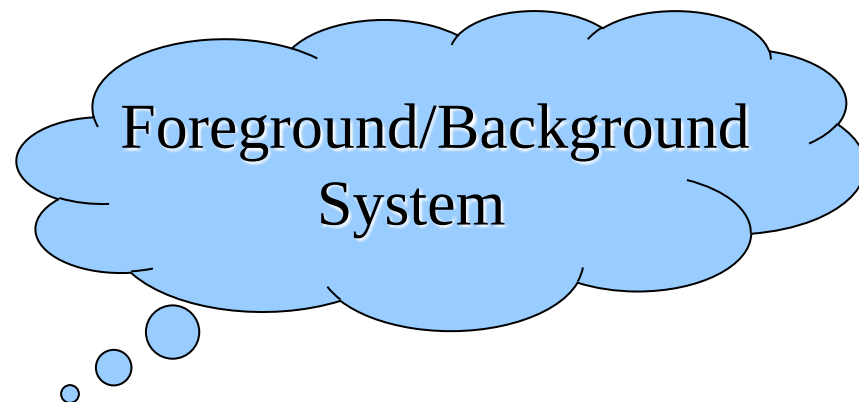


特点:

- (1) 响应及时 (**prompt response**)
- (2) 可靠性高 (**high reliability**)

1.4.4 通用操作系统(multi-purpose OS)

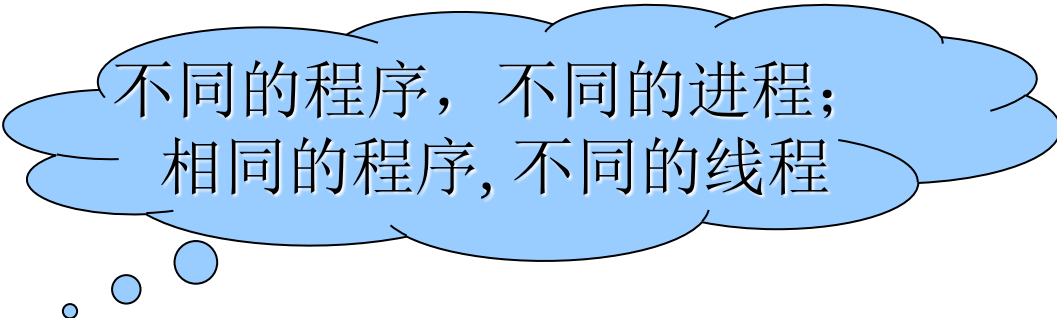
- 同时具有：分时、实时、批处理功能。
- 目标：
 - 提高处理能力;
 - 扩展应用领域。
- 常见模式：
 - 分时(前台)+批处理(后台) (eg. **DPS/8**上的**GCOS-8**)
 - 实时(前台)+批处理(后台)





1.4.5 单用户操作系统

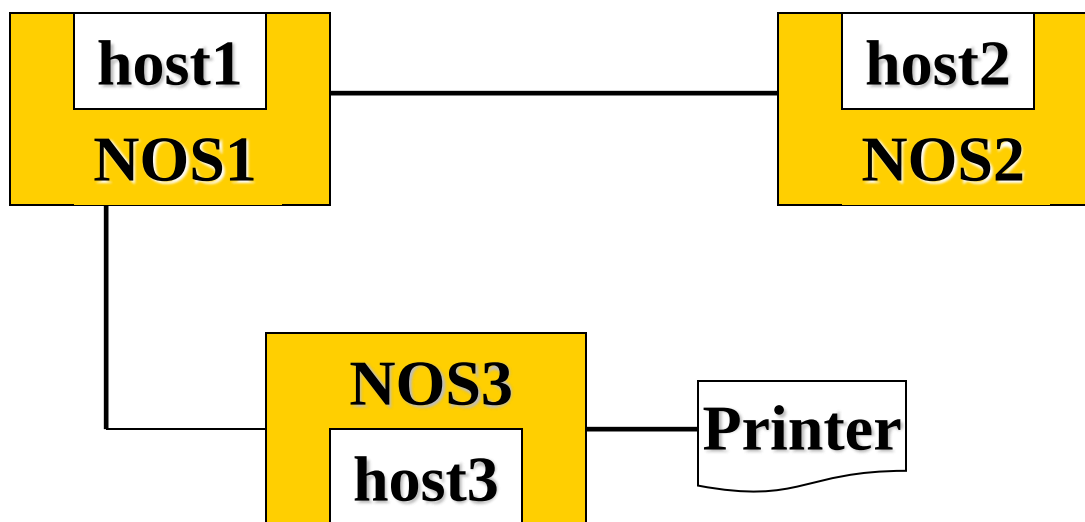
- 同一时刻仅有一个用户使用的系统
- 应用领域：
 - 台式机，笔记本，.....
- 特点：
 - 单用户，多进程，多线程

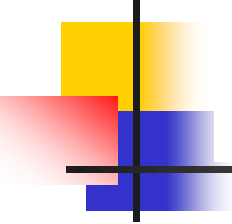


不同的程序，不同的进程；
相同的程序，不同的线程

1.4.6 网络操作系统(Network OS)

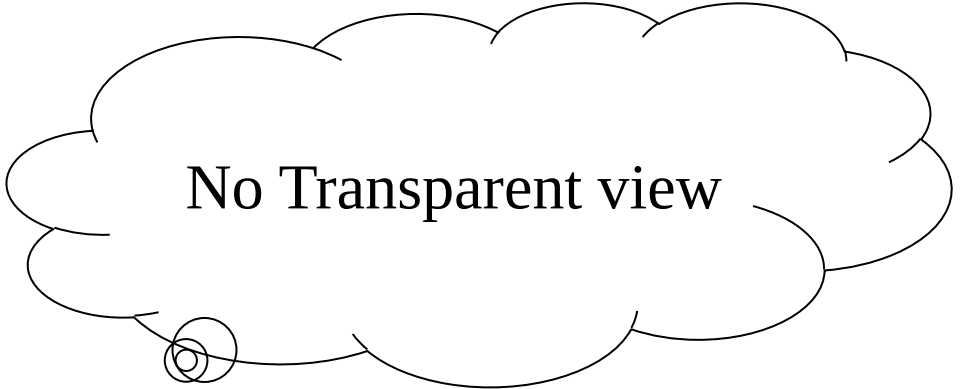
用于实现网络通信和网络资源管理的操作系统称为网络操作系统。





网络操作系统的目标

- 相互通讯
- 资源共享（信息，设备）
- 提供网络服务
 - **database server**
 - **ftp server**
 - **e-mail server**
 - **telnet server**
 - **etc.**



No Transparent view

1.4.7 分布式操作系统(Distributed OS)

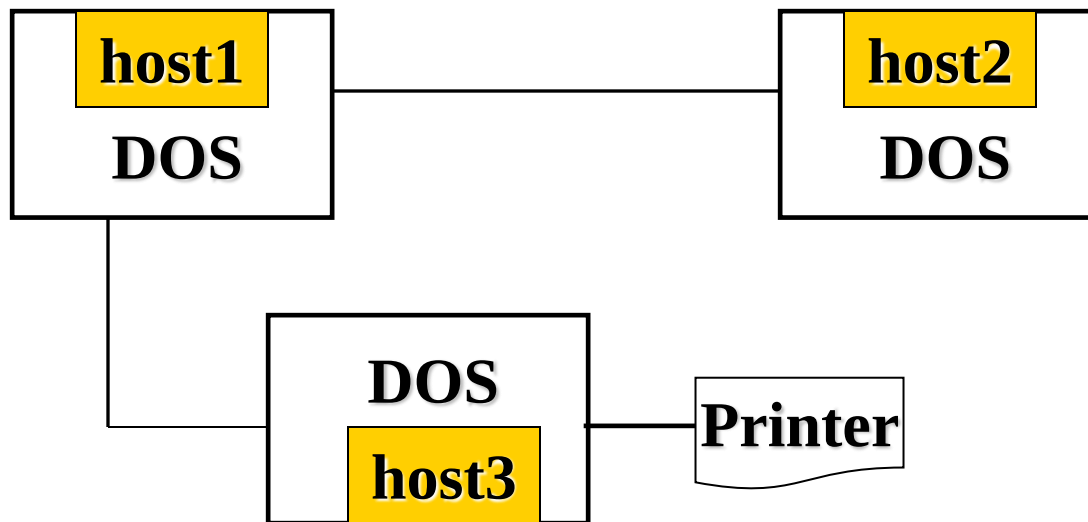
- 紧耦合: (**tightly coupled**)
 - 由多机系统发展而来 (多**CPU**)
 - 有公共内存
 - 多处理机操作系统



多处理机系统

1.4.7 分布式操作系统

- 松散耦合: (**loosely coupled**)
 - 由计算机网络发展而来 (多**Host**)
 - 无公共内存, 无公共时钟





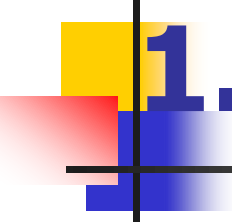
1.4.7 分布式操作系统(Cont.)

- 分布式操作系统特征:
 - 统一的操作系统
 - 资源的进一步共享
 - 内存, **CPU**
 - 可靠性
 - 透明性



1.4.7 分布式操作系统(Cont.)

- 目标：进一步共享资源，使负载均衡，计算加速。
 - CPU
 - 内存
- 途径：迁移（**migration**）
 - 作业迁移
 - 进程迁移（线程一般随同进程迁移）
- 例子：
 - Solaris MC



1.4.8 多处理机操作系统

- 多处理机系统
 - 具有公共内存的多**CPU**系统
- 对称多处理机系统(**SMP-symmetric multi-processor**)
 - 没有主从关系的多处理机系统
- 多处理机操作系统
 - 有效管理和使用多个**CPU**的操作系统
 - 特点：进程与**CPU**多对多
 - 新问题：
 - (1) 调度问题；(2) 并发控制问题
- 例子：
 - **UNIX, Linux, Windows**



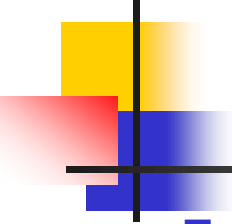
1.4.8 多处理机操作系统

- 优点：
 - 增加计算量
 - 规模经济：节省资金-共享外设、存储器、电源
 - 增加可靠性



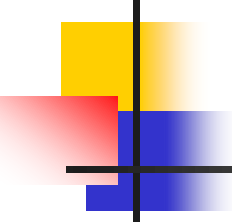
1.4.9 集群操作系统

- 集群操作系统建立在局域网络基础上，是指一种由多台计算机通过软件互相连接组成的并行或分布式系统。
- 集群的关键技术分为：网络层、终点机、操作系统层、集群系统管理层、应用层
- 分布式是指将不同的业务分布在不同的地方；而集群指的是将几台服务器集中在一起，实现同一业务；分布式结构中的每一个结点都可以做集群，而集群并不一定就是分布式的。



1.4.10 云计算操作系统

- 云的思想是把互联网中计算能力比较弱的各个结点统一管理起来。构成功能强大的计算机系统。
- 云计算操作系统又称云操作系统、云计算中心操作系统，是以云计算、云存储技术作为支撑的操作系统，是云计算后台数据中心的整体管理运营系统，它是架构于服务器、存储、网络等基础硬件资源和单机操作系统、中间件、数据库等基础软件之上的，管理海量的基础硬件和软件资源的云平台综合管理系统。



1.4.11 嵌入式操作系统

- 嵌入在掌上电脑、通讯设备、车载系统、信息家电等非计算机类设施上的操作系统。
- 特点：微内核结构（**Micro-kernel**），许多操作系统功能(文件系统,设备驱动)以应用程序模式运行。
 - 核心小(基本内存管理,**CPU**管理,通讯程序), 适应范围广, 可靠性高
 - 效率低
- 例子：
 - **Win CE .NET**（维纳斯, 美国微软）
 - **PalmOS**
 - **HOPEN**（女娲, 中科院钟锡昌）

Embedded world



1.4.11 嵌入式操作系统

■ 与一般操作系统的差别：

- 可裁剪性好：由于嵌入式系统的硬件配置和应用需求差别很大，所以要求嵌入式操作系统具备较好的适应性即可裁剪性，在配置较高、功能较多的环境中可以通过加载模块来满足需求；在配置较低、功能单一的环境中可以通过裁剪方式把一些不需要的模块裁减掉。
- 可移植性好：在嵌入式开发中，存在多种多样的**CPU**和底层硬件，设计系统时要考虑到软件的移植，使软件实现不同硬件平台的移植。
- 可扩展性：可以很容易地在原有的嵌入式系统上扩展新功能，这要求在系统设计时充分考虑功能之间的独立性，并未将来的扩展预留接口。



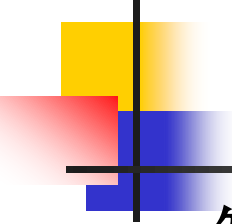
1.4.12 多媒体操作系统

■ 定义(百度百科)

- 具有一般操作系统功能；
- 还具有多媒体底层扩充模块，
支持多媒体信息的采集、编辑、播放和传输。

■ Remarks

- 不是一种独立的操作系统类型
- 是现代操作系统的一种特性
- 目前许多操作系统开始支持多媒体



1.4.13 智能卡操作系统

- 智能卡
 - CPU芯片
 - ROM
- 面向**Java**的智能卡
 - **JVM**解释程序
 - 下载**Java applet**并执行
- **SC-OS**
 - 支持多个**applet**并发执行
 - 必要的资源管理



1.5 操作系统运行环境

- 定时装置
- 堆与栈
- 寄存器
- 特权指令与非特权指令
- 处理机状态及状态转换
- 地址映射机构
- 存储保护设施
- 中断装置
- 通道与**DMA**控制器



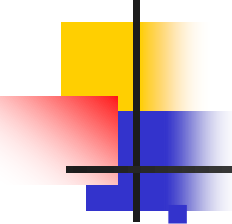
1.5.1 定时装置

- 绝对时钟：记载实际时间，不发中断。
 - 系统操作员可以修改，一般用户使用
 - 绝对时钟的值保存于硬件寄存器中
 - 程序可以读取绝对时钟的值



1.5.1 定时装置

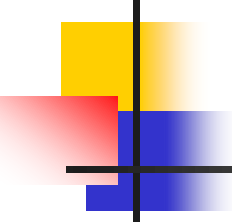
- 间隔时钟：定时发生中断，一般间隔单位为“毫秒”。
 - 间隔时钟是实现多道程序的基础—保证操作系统获得控制权。
 - 其它中断也进入操作系统，但是否发生，何时发生没有保障。
 - 通过间隔时钟可以构造逻辑时钟。



1.5.2 堆和栈(system stack)

在计算机科学领域中“堆”和“栈”是两种不同的数据结构，用途截然不同，尽管用户进程的“堆”和“栈”在物理上通常是相邻的。每个运行程序都有一个堆和两个栈（一个用户栈，一个系统栈）

- 堆属于用户空间，用于保存程序中的动态变量，例如树的结点，堆空间由操作系统分给运行程序，由于不同程序运行时对动态变量的使用不同，因而堆空间大小需求不定。
 - 操作系统为运行程序分配一个基本大小的堆，例如**8KB**，对堆空间的使用由应用软件管理
 - 若用户程序运行时对堆的需求超过了**8KB**，则会发生中断由操作系统为其扩充堆空间，如再分配**8KB**，，如此继续下去



1.5.2 堆和栈(system stack)

- 用户栈属于用户空间，用于保存用户函数调用时的返回点、参数、局部变量、返回值。除此之外，用户栈还要传送调用操作系统时传给操作系统的参数。
 - 用户程序调用操作系统时，有两个载体可以用来传递参数：
 - 寄存器：比较小的数据如：一个字符、一个整数、一个浮点数
 - 用户栈：比较长的参数如：文件名
 - 对每个系统调用，操作系统都规定了参数和返回值的存放位置，用户程序必须遵循相应的规定。



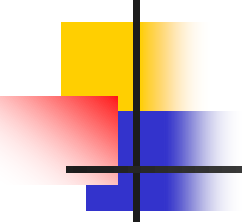
1.5.2 堆和栈(system stack)

- 系统栈也称为核心栈，逻辑上属于操作系统空间，程序切换的同时伴随着堆和用户栈以及系统栈的切换，但硬件的栈指针是多个进程共享的。
- 作用
 - 中断响应时保存中断现场
 - 保存函数调用返回点、参数、局部变量、返回值
- 数量
 - 每个可运行程序有一个对应的系统栈
 - 进程切换伴随着系统栈的切换
 - 系统栈的内容是变化的，但是硬件只有一个系统栈指针
- 位置
 - 内存中操作系统空间的一个固定区域



1.5.3寄存器

- 硬件系统提供一套寄存器，由运行程序使用。程序切换时，一般需把寄存器的当前值保存起来，再次运行前再恢复。
 - 程序状态字（**program status word,PSW**）：例如**IBM360/370**程序状态字由**16B**组成，表示当前程序的运行环境
 - 前**8**位是系统屏蔽位，**0-6**位对应通道，第**7**位对应外部中断
 - **12-15**位为**CMWP**，**13**位为开关中断为**M**，**15**位为系统状态位**P**
 - **16-31**位为中断码用于保存中断字
 - **36-38**位为程序屏蔽位，分别对应定点溢出、十进制溢出、阶下溢
 - **39**位为备用
 - 有些中断的是不可屏蔽的如：时钟、地址越界、缺页、非法指令
 - 指令计数器（**PC**）记载运行程序下一条指令的地址

- 
- 栈指针（**SP**）管态和目态各一个，分别保存系统栈和用户栈的栈顶位置
 - 通用寄存器（**regs**）若干个，用于存数和计算，还可以用来保存系统调用时传给操作系统的参数以及由操作系统传给用户的返回值
 - 浮点寄存器（**fregs**）若干个，用于存数和计算，还可以用来保存系统调用时传给操作系统的参数以及有操作系统传给用户的返回值
 - 地址映射寄存器，一般有一对，分别记录内存区域的起始地址和长度，分别称为基址寄存器（**base**）和限长寄存器(**limit**)



1.5.4 特权指令与非特权指令

- 特权指令(privileged instruction)
 - 只有在管态才能执行的指令(影响系统状态)
 - 开关中断, 置程序状态字, 停机, IO,
 - PSW: CPU运算器的一部分, 存放两类信息
 - 一类是体现当前指令执行结果的各类状态信息, 如有无进位、有无溢出、结果正负、结果是否为0等,
 - 一类是存放控制信息, 如允许中断等。
 - 特权指令只有操作系统才能执行, 用户程序不可执行
- 非特权指令(non-privileged instruction)
 - 在管态和目态都可以执行的指令(不影响系统状态)
 - 取数, 四则运算,



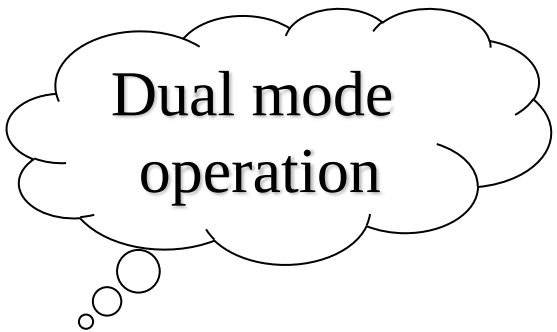
1.5.5 处理机状态及状态转换

■ 处理机状态

- 系统态 (**system mode**) (管态, 核态)
- 用户态 (**user mode**) (目态, 常态)

■ 状态转换

- 管态 $\Rightarrow$ 目态(置程序状态字, 特权指令)
- 目态 $\Rightarrow$ 管态(中断, **trap**)

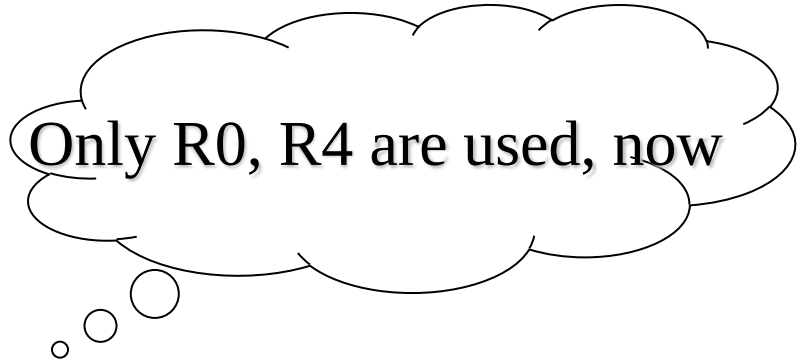


Dual mode
operation



例子:

- IBM 360/370 PSW 状态位(第15位)
 - 0:系统态
 - 1:用户态
- Modern PC now support 5 modes:
 - R0 (权限最强)
 - R1
 - R2
 - R3
 - R4 (权限最弱)

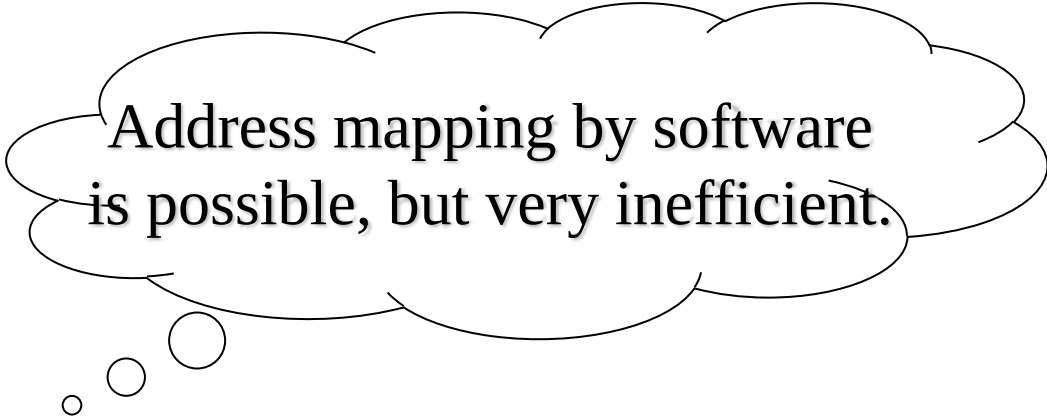


Only R0, R4 are used, now

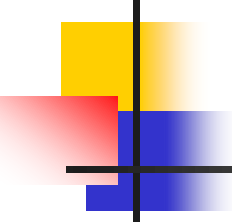


1.5.6 地址映射机构

- 逻辑地址 $\Rightarrow$ 物理地址
 - 逻辑地址(虚地址): 程序中产生的地址
 - 物理地址(实地址): 存储器地址



Address mapping by software is possible, but very inefficient.



1.5.7 存储保护设施

- 防止应用程序
 - 侵犯操作系统空间
 - 侵犯其它用户空间
 - 对共享区域的非法访问
- 地址检查
 - 越界检查;
 - 越权检查（对共享区域）



1.5.8 中断装置

- 发现并响应中断的硬件机构
 - 当前（**PSW**，**PC**） $\Rightarrow$ 系统栈
 - 中断向量（**PSW**，**PC**） $\Rightarrow$ 寄存器



1.5.9 通道与DMA

- 通道：接受**CPU**委托完成输入输出操作任务，即负责**IO**操作的处理机
 - 通道指令系统
 - 读写操作
 - 控制操作
 - 转移操作
 - 通道运控部件
 - 通道地址字**CAW**
 - 通道命令字**CCW**
 - 通道状态字**CSW**
 - 通道数据字**CDW**
- **DMA**：接受CPU委托直接依靠硬件完成数据在主存和块设备之间的传输
 - 没有独立指令系统
 - 简单块传输，一次只能传输一个块
 - 不可编程



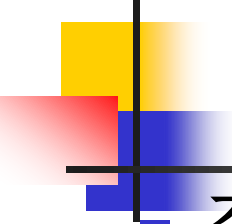
1.5.9 通道与DMA

- 相同点

- 都能实现IO设备和内存之间建立数据直传通路

- 不同点

- DMA只能实现固定的数据传送控制，而通道有自己的指令和程序，具有更强的独立处理数据输入和输出的能力。
- DMA只能控制一台或者少数几台同类设备，而一个通道可以控制多台同类或者不同的设备



1.6 操作系统界面形式

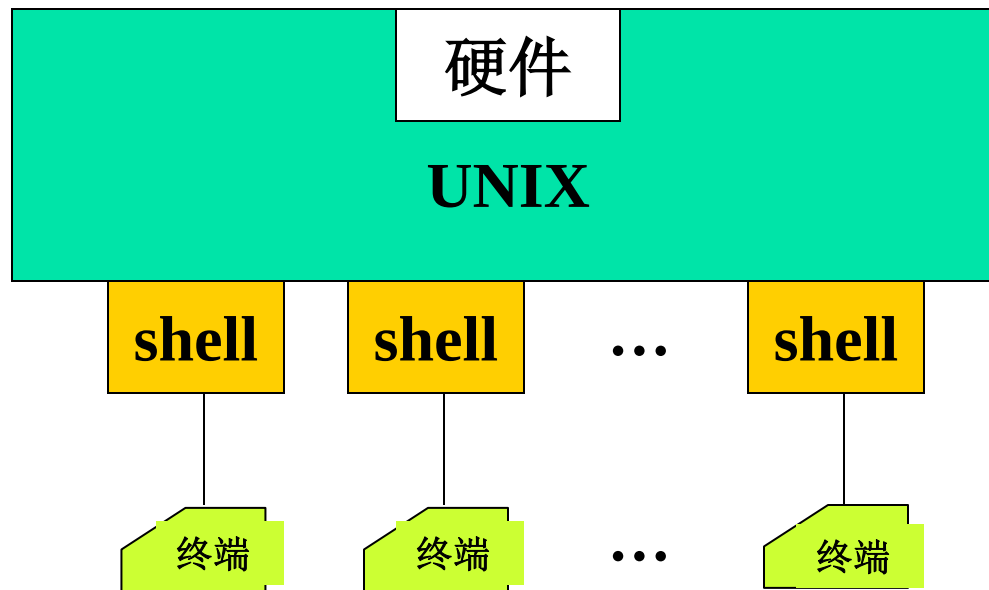
■ 交互终端命令（Command Language）

- **Eg. UNIX shell**

- **\$命令名 -选项 参数**

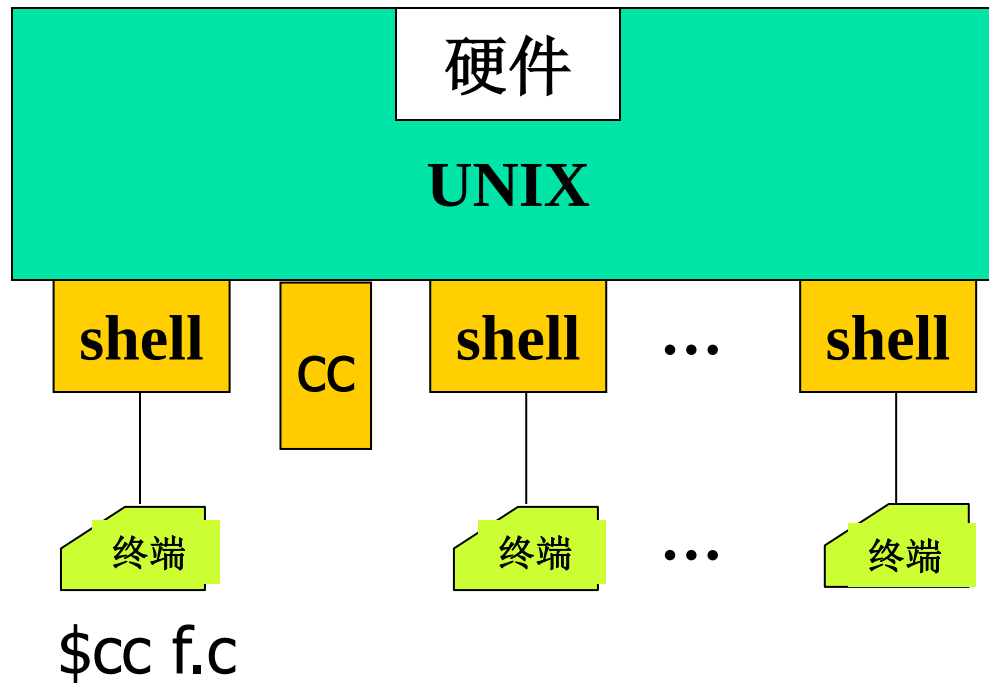
- 命令名指定操作功能
- 选项是对命令功能的调整
- 参数是命令操作的对象
- 例：\$rm -i *.c

UNIX shell interface



- **优点:**
 - 缩小核心
 - 不同用户可以选择不同界面

UNIX shell interface

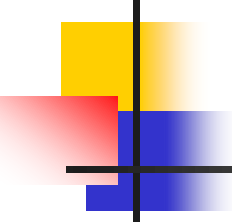


- **cc**与**shell**都属于目态进程
- 二者具有父子关系



1.6 操作系统界面形式(Cont.)

- 图形界面（GUI—Graphic User Interface）
 - 本质上属于交互式界面形式，只不过界面由命令行转变为图形提示和鼠标点击
 - 图形用户界面一般由视窗、图标、菜单、按钮、鼠标等基本元素以及对基本元素所能进行的操作构成
 - GUI的广泛应用是当今计算机发展的重大成就之一，它极大地方便了非专业用户的使用



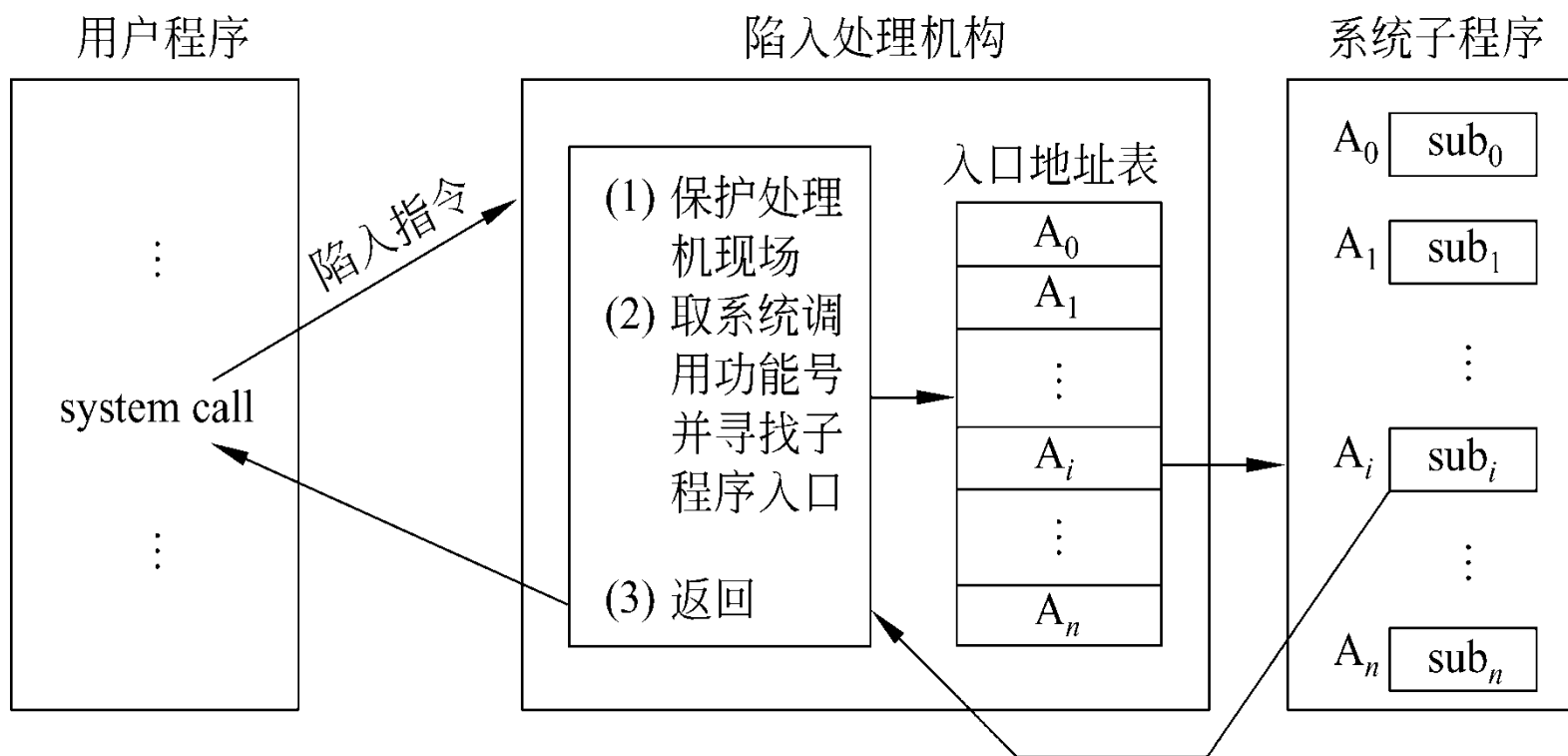
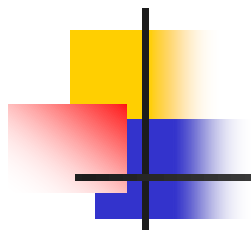
1.6 操作系统界面形式(Cont.)

- 作业控制语言 (Job Control Language)
 - 作业控制语言是用户与操作系统的接口
 - 用户通过作业控制语言的相应语句来与操作系统通讯，获得作业所需的资源等，按自己的意图来控制作业的执行
 - 批处理(普通命令的集合)操作系统界面形式
 - 作业是运行在后台的，不需要用户交互



1.6 操作系统界面形式(Cont.)

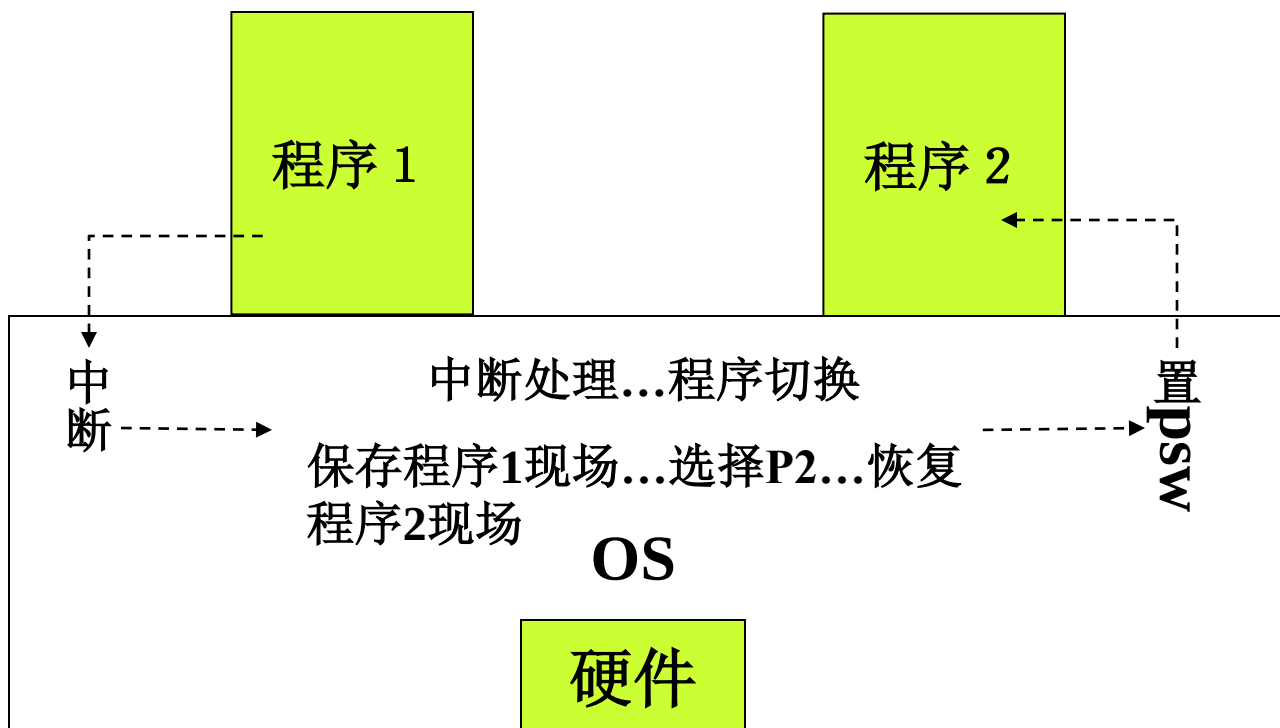
- 系统调用命令 (OS API)
 - 操作系统内核为应用程序提供的服务/函数
 - 系统调用是用户程序与操作系统打交道的方式，即应用程序请求操作系统核心完成某一特定功能的一种过程调用
 - 系统调用类型
 - 与文件相关的系统调用命令
 - 与进程相关的系统调用命令
 - 与进程通信相关的系统调用命令
 - 与资源相关的系统调用



系统调用的处理过程

1.7 操作系统的运行机理

■ 操作系统运行机理：





1.8 研究操作系统的几种观点

- 进程观点

- 将操作系统看成由若干个可以独立运行的程序和一个对这些程序进行协调管理的核心组成
- 同时运行的进程之间可能会发生相互作用(互斥、同步、通讯、死锁、饥饿等)
- 操作系统必须协调进程之间的相互作用, 以使各个进程正常结束并得到正确的结果



1.8 研究操作系统的几种观点

- 资源管理观点

- 操作系统是资源管理者：方便使用和防止冲突
- 为了管理资源，操作系统需要给出这些资源的状态描述，并基于资源状态描述给出相应的资源管理程序
- 用户在使用资源前需要向操作系统提出申请，用完后将资源归还



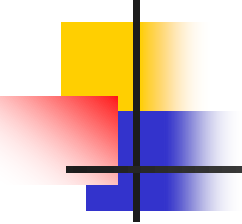
1.8 研究操作系统的几种观点

- 虚拟机观点
 - 认为操作系统是一个虚拟机，是硬件上运行的第一层软件
 - 提供虚拟资源
 - 将单个实的CPU→多个虚拟CPU
 - 内存+外存→虚拟存储
 - 独占型设备+共享型设备→虚拟设备



1.9 操作系统的功能

- 存储管理
 - 存储分配
 - 存储共享
 - 存储保护
 - 存储扩充
 - 地址映射
- 处理器管理
 - 进程控制
 - 进程同步与互斥
 - 进程通信
 - 处理器调度
- 文件管理
 - 文件的访问
 - 文件的组织
 - 目录管理
 - 文件的读、写管理与存取控制
- 设备管理
 - 设备的分配与去配
 - 缓冲管理
 - 设备驱动
 - 设备独立性
 - 虚拟设备
 - 设备调度



思考

- 应用程序运行需要操作系统提供哪些支持？

- `//Hello.c ->Hello.exe`

```
main()
```

```
{
```

```
    char *H="Hello";
```

```
    printf("%s", H);
```

```
    while(TRUE)
```

```
    {//死循环
```

```
        int i=100;
```

```
    }
```

```
}
```

(1) Hello.exe文件如何存放？

(2) Hello.exe程序如何启动？

(3) 如何为Hello.exe分配内存？

(4) printf如何输出字符串？

(5) while死循环会不会独占CPU？

(6) 程序结束如何退出系统？



操作系统的目标

- 提供计算机用户与计算机硬件之间的接口，使计算机系统更易于使用
- 有效地控制和管理计算机系统中的各种硬件和软件资源，使之得到更有效的利用
- 合理组织计算机系统的工作流程，以改善系统性能



系统举例

- UNIX系统
- Linux系统
- Windows系统
- HarmonyOS
- 麒麟OS
- 苹果MAC OS